CEE6501 — Lecture 9.2

Temperature Changes and Fabrication Errors

Learning Objectives

By the end of this lecture, you will be able to:

- Explain why temperature changes and fabrication errors generate internal forces in statically indeterminate structures
- Compute the local end displacement vector caused by a temperature gradient
- Derive local fixed-end force vectors due to temperature changes
- Distinguish between uniform temperature change and temperature gradient effects
- Compute fixed-end forces caused by fabrication errors in member length and member straightness
- Incorporate $\mathbf{q}_i^H$ and $\mathbf{q}_i^E$ into the DSM workflow

Agenda

- **Part 1** — Temperature Changes: Conceptual Framework
- **Part 2** — Temperature Changes: Simply Supported Member
- **Part 3** — Temperature Changes: Fixed-End Forces
- **Part 4** — Fabrication Errors: Conceptual Framework
- **Part 5** — Fabrication Errors: Fixed-End Force Expressions
- **Part 6** — DSM Implementation
- **Part 7** — Truss Example

Part 1 — Temperature Changes Conceptual Framework

Why Temperature Changes Matter

Like support settlements, temperature changes can induce significant stresses in a structure.

If a member were completely free, it would simply:

- expand or contract axially, and/or
- bend due to a temperature gradient through its depth.

But in a statically indeterminate structure, surrounding members and supports restrain these free deformations.

That restraint produces **member fixed-end forces**.

Connection to Support Settlement

The mechanics are very similar to support settlement:

- A member has some **free deformation** it would like to undergo
- The structure restrains that deformation
- Restraining it creates a **fixed-end force vector**

The main difference is the coordinate system in which the effect is most naturally described:

- **Support settlement** is usually prescribed in the **global** system
- **Temperature changes** are usually defined in the **local member** system
- **Fabrication errors** are also usually defined in the **local member** system

Core Idea

We first determine the **free local end displacements** caused by the temperature change.

Then we ask:

> *What local end forces would be required to suppress those displacements?*

That force vector is the local thermal fixed-end force vector:

$$\mathbf{q}^H$$

This is exactly the same superposition logic used earlier for member loads and support settlement.

Notation

H superscript indicates FEFs due to temperature change.

Assume the start node is b and the end node is e.

$$\mathbf{q}^H = \begin{bmatrix} q_1^H \\ q_2^H \\ q_3^H \\ q_4^H \\ q_5^H \\ q_6^H \end{bmatrix} = \begin{bmatrix} q_{xb}^H \\ q_{yb}^H \\ q_{mb}^H \\ q_{xe}^H \\ q_{ye}^H \\ q_{me}^H \end{bmatrix} = \begin{bmatrix} N_b^H \\ V_b^H \\ M_b^H \\ N_e^H \\ V_e^H \\ M_e^H \end{bmatrix}$$

We write local fixed-end forces using the convention:

- q_{xb}^H = FEF at node **b** in local **x** (axial) → corresponds to **DOF 1**
- q_{yb}^H = FEF at node **b** in local **y** (shear) → corresponds to **DOF 2**
- q_{mb}^H = FEF at node **b** (moment) → corresponds to **DOF 3**
- q_{xe}^H = FEF at node **e** in local **x** (axial) → corresponds to **DOF 4**
- q_{ye}^H = FEF at node **e** in local **y** (shear) → corresponds to **DOF 5**
- q_{me}^H = FEF at node **e** (moment) → corresponds to **DOF 6**

So the fixed-end force vector is

$$\mathbf{q}^{H,T} = \begin{bmatrix} q_{xb}^H & q_{yb}^H & q_{mb}^H & q_{xe}^H & q_{ye}^H & q_{me}^H \end{bmatrix}^T$$

Part 2 — Temperature Changes: Simply Supported Member

Temperature Field Assumption

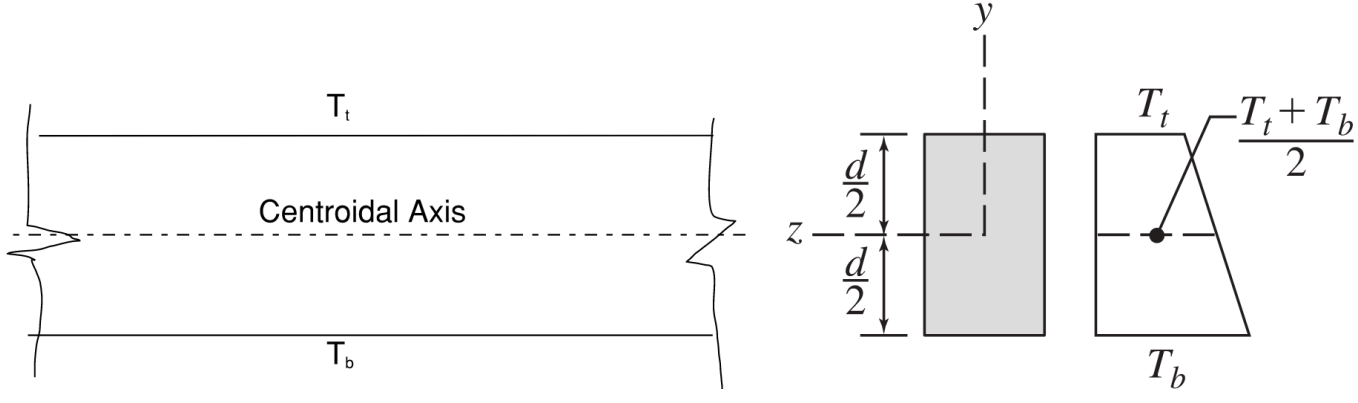
Consider a plane-frame member of length L and depth d.

Assume:

- top surface temperature increase = T_t
- bottom surface temperature increase = T_b
- temperature varies **linearly through the depth**
- temperature does **not** vary along the member length
- coefficient of thermal expansion = α

We assume the member is initially **simply supported**, so it is free to deform without developing end forces.

Temperature Change over Member Depth



Axial Elongation of the Centroidal Axis

The temperature increase at the centroidal axis is the average of the top and bottom values:

$$\frac{T_t + T_b}{2}$$

So the axial elongation of the member is

$$\Delta_H = \alpha \left(\frac{T_t + T_b}{2} \right) L$$

where α is the coefficient of thermal expansion, with units of strain per degree of temperature change (for example, $/^\circ\mathrm{C}$).

This quantity tells us how much a material wants to expand or contract per unit length for each degree of temperature change.

Thus, Δ_H is the **free axial expansion** caused by the **average temperature increase** across the member depth.

Typical values of the coefficient of thermal expansion:

- **Concrete**: $\alpha \approx (8\text{–}12) \times 10^{-6}\ /^{\circ}\mathrm{C}$
- **Structural steel**: $\alpha \approx 12 \times 10^{-6}\ /^{\circ}\mathrm{C}$
- **Wood (Parallel to grain)**: $\alpha \approx (3\text{–}5) \times 10^{-6}\ /^{\circ}\mathrm{C}$
- **Wood (Perpendicular to grain)**: $\alpha \approx (30\text{–}60) \times 10^{-6}\ /^{\circ}\mathrm{C}$
- **Aluminum** $\alpha \approx (22\text{–}24) \times 10^{-6}\ /^{\circ}\mathrm{C}$

End Rotations Due to Temperature Gradient

If $T_b \neq T_t$, the top and bottom fibers elongate by different amounts.

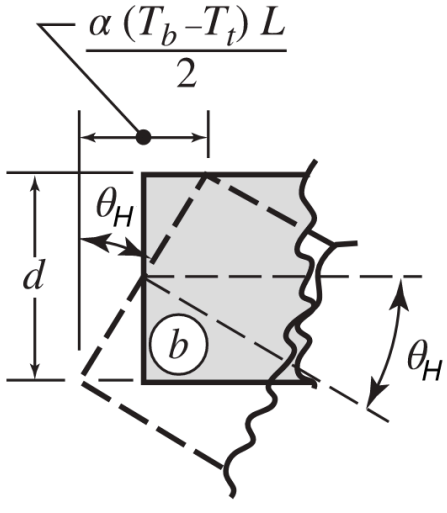
That causes the member to curve.

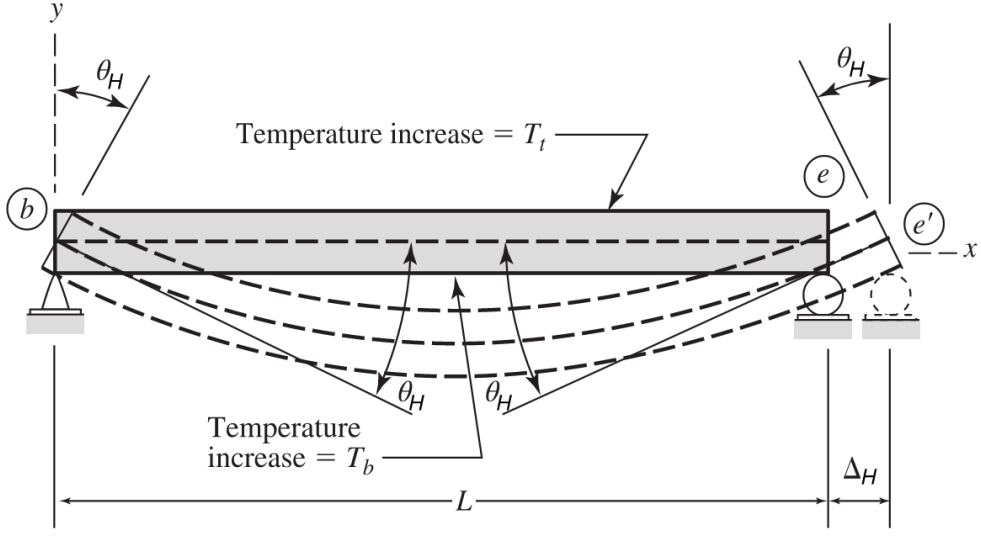
The magnitude of the end rotation (assuming θ is small) is

$$\theta_H \approx \frac{\Delta_b - \Delta_t}{d} = \frac{\alpha(T_b - T_t)L}{2d}$$

Using our sign convention:

- rotation at the **begin node** b is clockwise, so it is **negative**
- rotation at the **end node** e is counterclockwise, so it is **positive**


$\frac{\alpha (T_b - T_t) L}{2}$
θ_H
d
b
θ_H


y
θ_H
θ_H
Temperature increase = T_t
b
e
e'
x
θ_H
θ_H
Temperature increase = T_b
Δ_H
L

Local End Displacement Vector Due to Temperature

Using the local frame element DOF ordering

$$\mathbf{v}^T = \begin{bmatrix} v_1 & v_2 & v_3 & v_4 & v_5 & v_6 \end{bmatrix}^T = \begin{bmatrix} v_{xb} & v_{yb} & \theta_b & v_{xe} & v_{ye} & \theta_e \end{bmatrix}^T$$

the free local end displacement vector, for a simply supported beam, due to temperature change is

$$\mathbf{v}^H = \begin{bmatrix} 0 \\ 0 \\ -\theta_H \\ \Delta_H \\ 0 \\ \theta_H \end{bmatrix}$$

where v_3 is negative because the rotation is clockwise at the starting node

Reading the Temperature Displacement Vector

$$\mathbf{v}^H = \begin{bmatrix} 0 & 0 & -\theta_H & \Delta_H & 0 & \theta_H \end{bmatrix}^T$$

Interpretation:

- no transverse joint translations: $v_2 = v_5 = 0$
- no axial motion at the start node: $v_1 = 0$
- axial expansion appears at the end node: $v_4 = \Delta_H$
- equal and opposite end rotations appear due to curvature: $v_3 = -\theta_H$ and $v_5 = \theta_H$

This is a **free deformation vector** for the simply supported member.

substituting the expressions for Δ_H and θ_H,

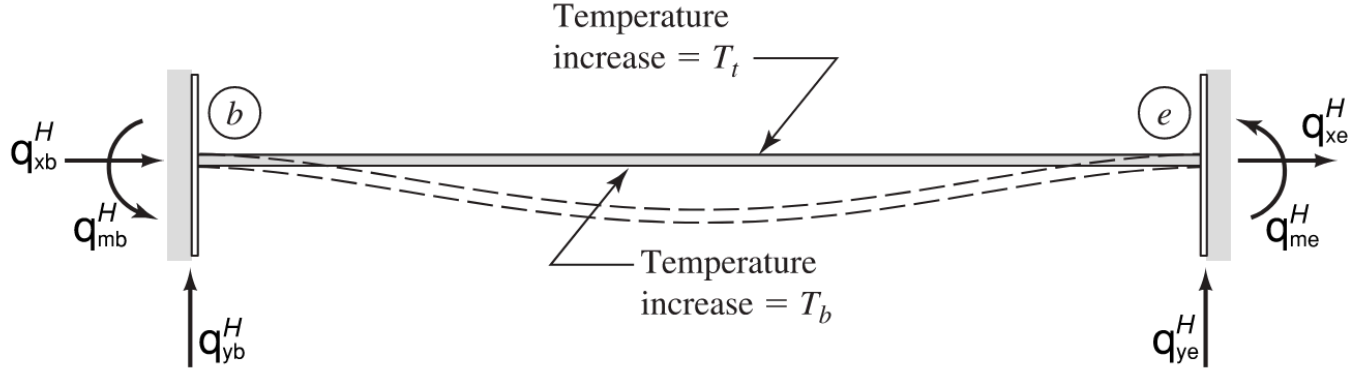
$$\mathbf{v}^H = \frac{\alpha L}{2} \begin{bmatrix} 0 \\ 0 \\ -(T_b - T_t)/d \\ T_b + T_t \\ 0 \\ (T_b - T_t)/d \end{bmatrix}$$

Part 3 — Temperature Changes: Equivalent Fixed-End Forces

Superposition Process

(1) Fixed–Fixed Member: Temperature Change

A prismatic frame member is restrained against end displacements and rotations. A uniform temperature increase induces **thermal strain**, which is prevented by the supports, generating **fixed-end forces**.

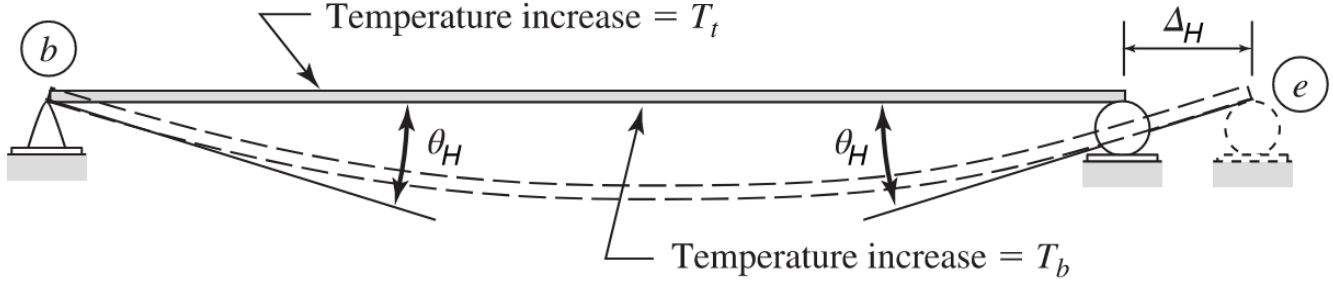


Note: all q forces are shown in their positive direction, based on sign convention, not the actual direction under axial expansion.

(2) Simply Supported Member: Temperature Change Only

The same member is now simply supported and subjected to the same temperature increase.

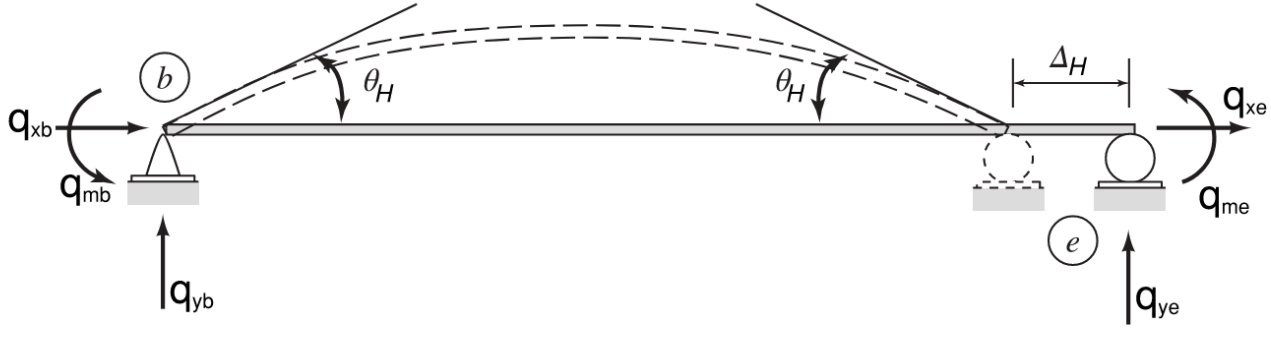
- Ends are free to expand
- **No end forces develop**
- Thermal expansion produces end displacements, v^H



(3) Simply Supported Member: End Forces Only

Apply the **fixed-end forces** from Case (1), but with **no temperature change**.

- Mechanical loading only
- Produces end displacements opposite to thermal expansion



Note: all q forces are shown in their positive direction, based on sign convention, not the actual direction under axial expansion.

Superposition Principle

The restrained (fixed–fixed) member response equals the sum of two simply supported cases:

$$(1)\ \text{Fixed member} = (2)\ \text{Temperature-only} + (3)\ \text{End-force-only}$$

Displacement form:

$$\mathbf{v}^{(1)} = \mathbf{v}^{(2)} + \mathbf{v}^{(3)}$$

End displacements:

- Fixed member restrained: $\mathbf{v}^{(1)} = \mathbf{0}$
- Simply supported + temperature (free expansion): $\mathbf{v}^{(2)} = \mathbf{v}^{H}$

Compatibility:

$$\mathbf{0} = \mathbf{v}^{H} + \mathbf{v}^{(3)} \quad \Rightarrow \quad \mathbf{v}^{(3)} = -\mathbf{v}^{H}$$

Computing Thermal Fixed-End Forces

Thus, the end forces in (3) that cancel thermal expansion in (2) are obtained from the local stiffness relation:

$$\mathbf{q}^H = \mathbf{k}\left(-\mathbf{v}^H\right)$$

- $\mathbf{q}^H$ — thermal fixed-end force vector
- $\mathbf{v}^H$ — thermal end displacement vector
- $\mathbf{k}$ — local element stiffness matrix

Superposition Summary

To obtain the fixed-end forces for the restrained member:

1. Consider the simply supported member subjected to the temperature change
2. It develops the free end displacements $\mathbf{v}^H$ but no end forces
3. Apply end forces that produce the opposite displacement vector $-\mathbf{v}^H$

Those end forces are exactly the fixed-end forces required to suppress the free thermal deformation.

Local FEFs from the Local Stiffness Matrix

$$\mathbf{q}^H = \mathbf{k}\left(-\mathbf{v}^H\right)$$

$$= \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix} \left(-\frac{\alpha L}{2} \begin{bmatrix} 0 \\ 0 \\ -(T_b - T_t)/d \\ T_b + T_t \\ 0 \\ (T_b - T_t)/d \end{bmatrix} \right)$$

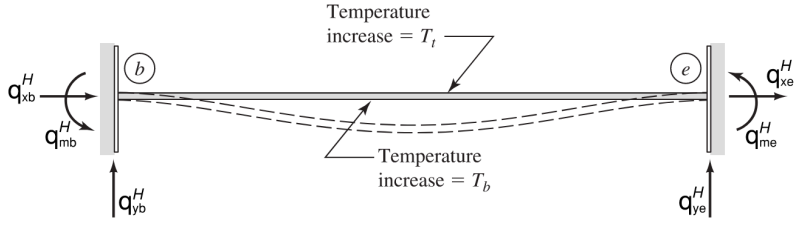
Local FEFs Vector

Assume the begin node is b and the end node is e.

$$\mathbf{q}^H = \begin{bmatrix} q_{xb}^H \\ q_{yb}^H \\ q_{mb}^H \\ q_{xe}^H \\ q_{ye}^H \\ q_{me}^H \end{bmatrix} = E\alpha \begin{bmatrix} A(T_b + T_t)/2 \\ 0 \\ I(T_b - T_t)/d \\ -A(T_b + T_t)/2 \\ 0 \\ -I(T_b - T_t)/d \end{bmatrix}$$

Component Form

Note: all q forces are shown in their positive direction



The thermal fixed-end force expressions are therefore

$$q^H_{xb} = -q^H_{xe} = EA\alpha\left(\frac{T_b + T_t}{2}\right)$$

$$q^H_{yb} = q^H_{ye} = 0$$

$$q^H_{mb} = -q^H_{me} = EI\alpha\left(\frac{T_b - T_t}{d}\right)$$

Special Case: Uniform Temperature Change

If the temperature increase is uniform through the depth,

$$T_b = T_t = T_u$$

then the gradient vanishes, so the bending terms disappear.

The fixed-end force vector becomes

$$\mathbf{q}^H = EA\alpha T_u \begin{bmatrix} 1 \\ 0 \\ 0 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

Uniform temperature change causes axial restraint force, but no bending moment.

Beams, Frames, and Trusses

These expressions adapt naturally by member type:

- **Plane frame member**: both axial and bending terms may be present
- **Beam member**: axial expansion is free, so only the moment terms matter
- **Truss member**: member ends are free to rotate, so only the axial terms matter

In particular:

- For a **beam**: $q_{xb}^H = 0$ and $q_{xe}^H = 0$
- For a **truss**: $q_{mb}^H = 0$ and $q_{me}^H = 0$

Part 4 — Fabrication Errors Conceptual Framework

What Is a Fabrication Error?

A fabrication error is a small initial deformation or geometric imperfection of a member in its unstressed state.

Common examples include:

- the member being fabricated too long or too short
- the member not being perfectly straight

If the structure must force that imperfect member into the intended geometry, internal forces develop.

Same Superposition Logic as Temperature

The derivation is parallel to the thermal case:

1. Determine the member's free end displacements caused by the geometric error, v^E
2. Apply equal and opposite end actions to suppress those displacements, $-v^E$
3. The required end loads are the fabrication-error fixed-end forces

We will denote these by

$$\mathbf{q}^E$$

Two Common Types of Fabrication Error

In this lecture we consider:

1. Error in initial member length, a
2. Error in initial member straightness, b

These lead to different forms of $\mathbf{q}^E$

Part 5 — Fabrication Errors: Fixed-End Force Expressions

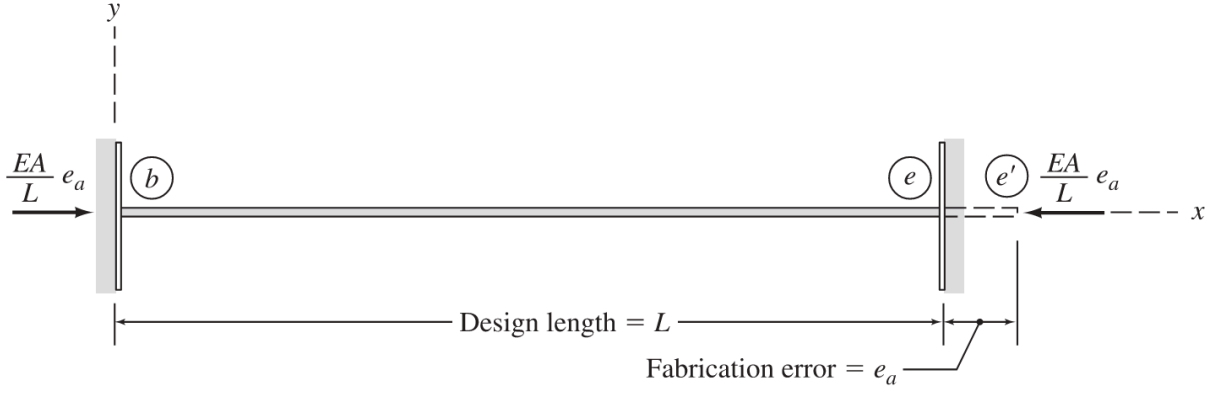
Fabrication Error in Member Length

Suppose the design length is L, but the unstressed fabricated length is

$$L + e_a$$

with $e_a > 0$ meaning the member is too long.

To fit the member between supports separated by distance L, the structure must compress it axially.



Local End Displacement Vector Due to Member Length

For a beam/frame member, the local end displacement vector due to the extra length is

$$\mathbf{v}^{E,a} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ e_a \\ 0 \\ 0 \end{bmatrix} = e_a \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Local FEFs from the Local Stiffness Matrix

Apply end forces that produce the opposite displacement vector: $-\mathbf{v}^{E,a}$

$$\mathbf{q}^{E,a} = \mathbf{k}\left(-\mathbf{v}^{E,a}\right)$$

$$= \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix} \left(-e_a \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \right)$$

That induces the local fabrication-error fixed-end force vector

$$\mathbf{q}^{E,a} = \frac{EA}{L} e_a \begin{bmatrix} 1 \\ 0 \\ 0 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

Component Form for Length Error

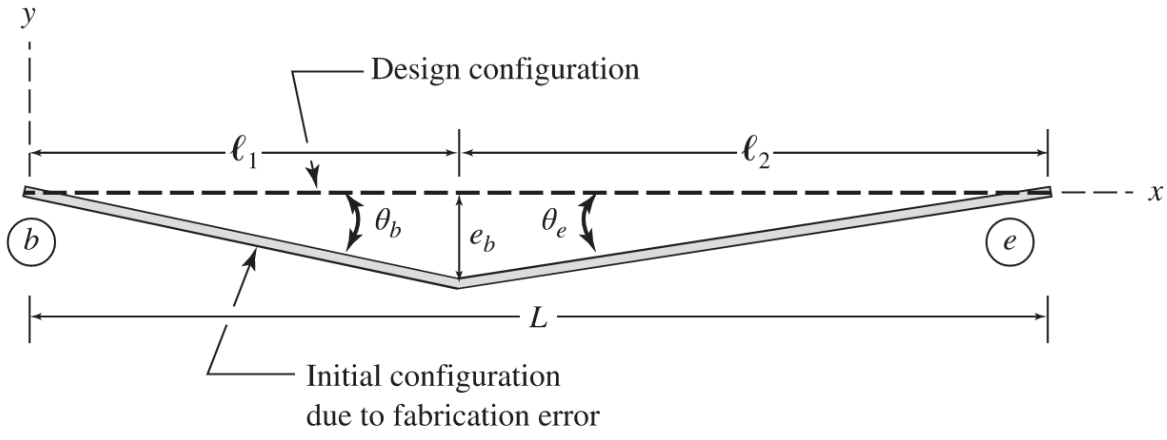
$$q_{xb}^{E,a} = -q_{xe}^{E,a} = \frac{EA}{L} e_a$$

$$q_{yb}^{E,a} = q_{ye}^{E,a} = 0$$

$$q_{mb}^{E,a} = q_{me}^{E,a} = 0$$

This expression also applies directly to truss members, since the effect is purely axial.

Error in Initial Member Straightness



Let the initial straightness error be e_b, occurring at a point that divides the member into lengths l_1 and l_2, with

$$L = l_1 + l_2$$

Assuming small angles, then the corresponding end rotations are

$$\theta_b \approx \frac{e_b}{l_1} \qquad \theta_e \approx \frac{e_b}{l_2}$$

Using our local sign convention:

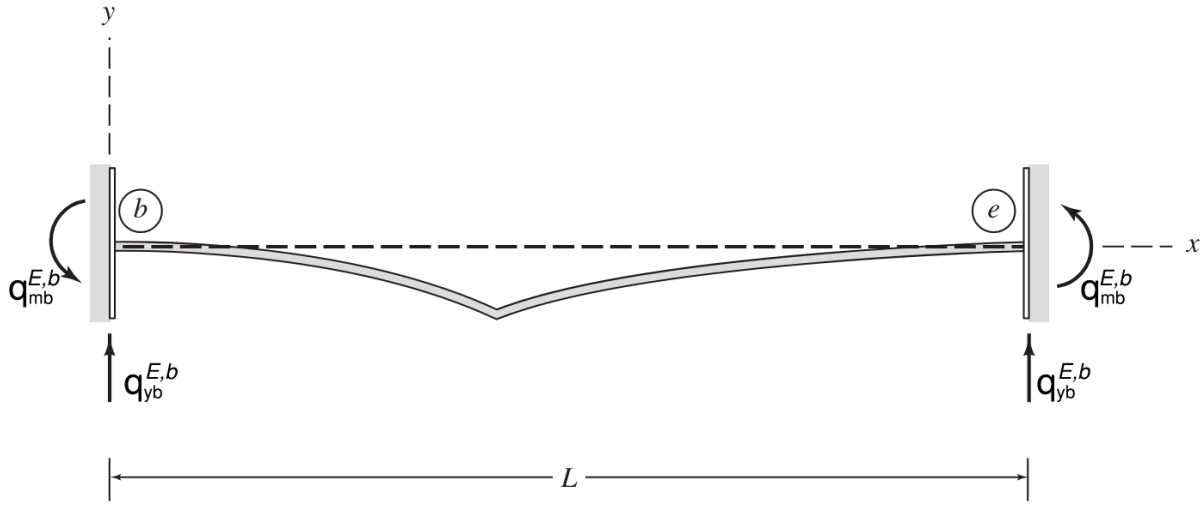
- the begin-end rotation is clockwise, so it enters as negative
- the end-node rotation is counterclockwise, so it enters as positive

Local End Displacement Vector Due to Straightness Error

For a beam/frame member, the local end displacement vector due to the initial bend is

$$\mathbf{v}^{E,b} = \begin{bmatrix} 0 \\ 0 \\ -\theta_b \\ 0 \\ 0 \\ \theta_e \end{bmatrix} = e_b \begin{bmatrix} 0 \\ 0 \\ -1/l_1 \\ 0 \\ 0 \\ 1/l_2 \end{bmatrix}$$

Apply end forces that produce the opposite displacement vector $-\mathbf{v}^{E,b}$



Note: all q forces are shown in their positive direction

Local FEFs from the Local Stiffness Matrix

$$\mathbf{q}^{E,b} = \mathbf{k}\left(-\mathbf{v}^{E,b}\right)$$

$$= \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix} \left(-e_b \begin{bmatrix} 0 \\ 0 \\ -1/l_1 \\ 0 \\ 0 \\ 1/l_2 \end{bmatrix} \right)$$

Local FEFs Vector

Assume the begin node is b and the end node is e.

$$\mathbf{q}^{E,b} = \begin{bmatrix} 0 \\ \dfrac{6EIe_b}{L^2 l_1 l_2}(l_2 - l_1) \\ \dfrac{2EIe_b}{L l_1 l_2}(2l_2 - l_1) \\ 0 \\ -\dfrac{6EIe_b}{L^2 l_1 l_2}(l_2 - l_1) \\ \dfrac{2EIe_b}{L l_1 l_2}(l_2 - 2l_1) \end{bmatrix}$$

Component Form for Straightness Error

$$q_{xb}^{E,b} = q_{xe}^{E,b} = 0$$

$$q_{yb}^{E,b} = -q_{ye}^{E,b} = \frac{6EIe_b}{L^2 l_1 l_2}(l_2 - l_1)$$

$$q_{mb}^{E,b} = \frac{2EIe_b}{Ll_1 l_2}(2l_2 - l_1)$$

$$q_{me}^{E,b} = \frac{2EIe_b}{Ll_1 l_2}(l_2 - 2l_1)$$

Since the straightness error is assumed small, it produces no axial deformation, only bending-type restraint effects.

Total Fabrication-Error FEFs

If multiple fabrication-error sources are present, their local fixed-end force vectors add:

$$\mathbf{q}^E = \mathbf{q}^{E,a} + \mathbf{q}^{E,b} + \cdots$$

So fabrication error is treated just like any other equivalent nodal load source.

Part 6 — DSM Implementation

Where Thermal and Fabrication FEFs Enter the DSM Equations

At the element level,

$$\mathbf{q}_i = \mathbf{k}_{l,i}\mathbf{v}_i + (\mathbf{q}_i^F + \mathbf{q}_i^H + \mathbf{q}_i^E)$$

At the assembled global level,

$$\mathbf{f} = \mathbf{K}\mathbf{u} + (\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$$

or, equivalently,

$$\mathbf{K}\mathbf{u} = \mathbf{f} - (\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$$

So thermal and fabrication effects enter the system exactly like other equivalent nodal force terms.

Why Do FEF Terms Appear with a Minus Sign?

$$\mathbf{K}\mathbf{u} = \mathbf{f} - \left(\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E\right)$$

The left-hand side (LHS), $\mathbf{K}\mathbf{u}$, represents the **nodal forces generated by structural deformation**.

When we solve for $\mathbf{u}$, we are finding the displacement vector that produces the additional deformation-induced forces required for equilibrium.

The right-hand side (RHS), $\mathbf{f} - \left(\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E\right)$, represents the **net applied nodal load still to be balanced** by the LHS after the known fixed-end-force contributions have already been included.

Physical Interpretation

A member load, temperature change, or fabrication error gives the element a **free deformation** it would naturally like to undergo.

If that free deformation is restrained, forces develop at the element ends. Those are the fixed-end forces, $(\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$

In the assembled system, the fixed-end-force contribution appears at each structural node as an equal-and-opposite nodal load required by equilibrium, which is why it enters with a minus sign.

$$-(\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$$

So before we solve for the structural displacements, $\mathbf{u}$, the elements in the structure already contribute a known force pattern.

The term $\mathbf{K}\mathbf{u}$ then provides only the **additional** force needed so that the structure satisfies compatibility and equilibrium.

Takeaway

The minus sign does **not** mean the FEFs are inherently negative.

The fixed-end-force terms appear with a minus sign because they are **known force contributions already present in the structure**.

They reduce the amount of force that must be generated by deformation, so the displacement solution $\mathbf{Ku}$ balances only the **remaining unbalanced load**.

$$\mathbf{Ku} = \mathbf{f} - (\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$$

Implementation Steps

1. Build each element local stiffness matrix $\mathbf{k}_{l,i}$
2. Compute any element-level vectors $\mathbf{q}_i^F$, $\mathbf{q}_i^H$, $\mathbf{q}_i^E$
3. Transform to global coordinates
4. Assemble the global equivalent nodal force vectors
5. Solve the global system

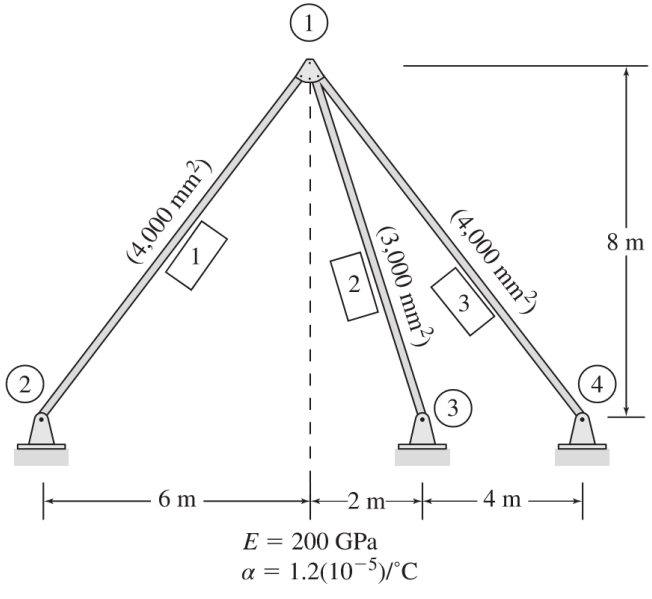
$$\mathbf{K}\mathbf{u} = \mathbf{f} - (\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$$

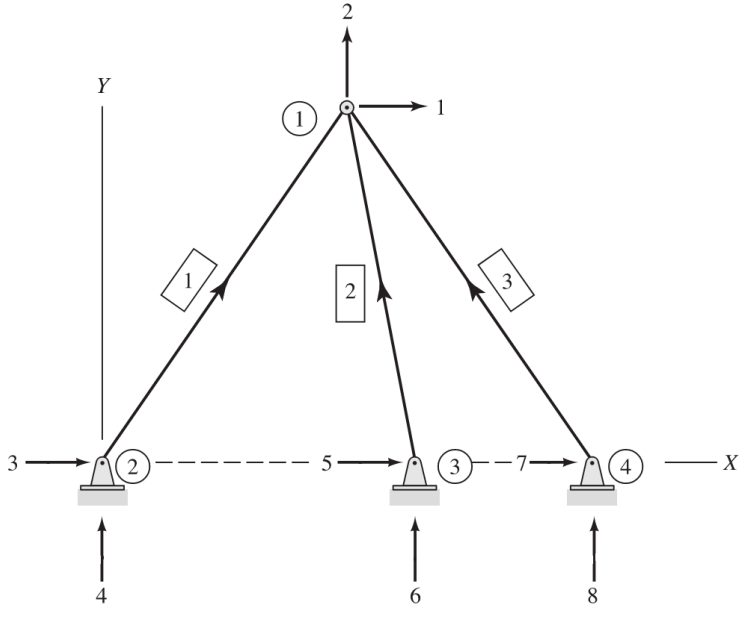
6. Perform the backward pass to recover local element end forces

Part 7 — Truss Example

Example Problem (7.5 Kassimali)

Same truss structure we looked at for support settlement





Determine the joint displacements, member axial forces, and support reactions for the plane truss due to the combined effect of the following:

(a) Joint loading

- Nodal load at Node 1:

$$\mathbf{P}_1 = [600,\ -900]\ \mathrm{kN}$$

(b) Thermal effect

- Uniform temperature decrease of $15^\circ\mathrm{C}$ in **Member 1**

(c) Fabrication error

- **Member 3** fabricated $3\ \mathrm{mm}$ too short

In [1]:

```python
# -------------------------
# Import Libraries and Helper Functions
# -------------------------
# This slide imports the required Python libraries and adds the
# course helper-code directory to the Python path.

import sys
import os
import math
import numpy as np

# Get current notebook directory
current_dir = os.getcwd()

# Go up two levels, then into Code/L8
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L8")
)

sys.path.append(helpers_path)
```

In [2]:
```
from helper_funcs_dsm import (
    assemble_global_stiffness_and_fef,
    partition_system,
    assemble_global_displacements,
    assemble_global_forces
)
from helper_funcs_output import (
    print_element_truss,
    print_dsm_results,
    print_matrix_scaled,
)

# -------------------------
# Problem data (hard-coded)
# -------------------------
E = 200e6          # kN/m^2
alpha = 1.2e-5     # 1/°C
```

In [3]:

```
# -------------------------
# Member 1, hard-coded
# -------------------------
A1 = 0.004 # m^2
L1 = math.sqrt(6**2 + 8**2)

theta = math.atan2(8,6)  # radians
c = np.cos(theta)
s = np.sin(theta)

factor = E * A1 / L1

k1 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

T1 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

# Mapping
m1 = np.array([3,4,1,2])
```

In [4]:
```
# Temperature change in truss member 1
T_b = -15
T_t = -15

Qf1 = E * alpha * np.array([
    A1 * (T_b + T_t) / 2,
    0.0,
    -A1 * (T_b + T_t) / 2,
    0.0,
])

print(Qf1)
```

```
[-144.    0.  144.    0.]
```

In [5]:

```
# -------------------------
# Member 2, hard-coded
# -------------------------
A2 = 0.003 # m^2
L2 = math.sqrt(2**2 + 8**2)

theta = math.atan2(8,-2)
c = np.cos(theta)
s = np.sin(theta)

factor = E * A2 / L2

k2 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

T2 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

m2 = np.array([5,6,1,2])
```

In [6]:
```
# Local fixed-end vector for member 2
Qf2 = np.zeros(4, dtype=float)
```

In [7]:

```
# -------------------------
# Member 3, hard-coded
# -------------------------
A3 = 0.004 # m^2
L3 = math.sqrt(6**2 + 8**2)

theta = math.atan2(8,-6)
c = np.cos(theta)
s = np.sin(theta)

factor = E * A3 / L3

k3 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

T3 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

m3 = np.array([7,8,1,2])
```

In [8]:
```
# Fabrication error in member 3
e_a = -3/1000 # mm -> m

Qf3 = E * A3 / L3 * np.array([
    e_a,
    0.0,
    -e_a,
    0.0,
])

print(Qf3)
```

```
[-240.    0.  240.    0.]
```

In [9]:
```
# -------------------------
# Initialize Global System
# -------------------------
# This slide sets up the global variables needed for assembly of the
# Direct Stiffness Method system.

k_list = [k1, k2, k3]
T_list = [T1, T2, T3]
Qf_list = [Qf1, Qf2, Qf3]
map_list = [m1, m2, m3]   # 1-based DOF maps

# Global system size (still using 1-based maps here)
ndof = int(np.max(np.concatenate(map_list)))

# Initialize Applied Load
F_global = np.zeros(ndof, dtype=float)

# Initialize Global Displacement
u_global = np.zeros(ndof, dtype=float)
```

In [10]:

```
# -------------------------
# User-Specified Boundary Conditions
# -------------------------
# This slide defines the external loads and support conditions.

# Global Forces (0-indexed)
F_global[1-1] = 600 #kN
F_global[2-1] = -900 #kN

dof_restrained_1based = np.sort(
    np.array([3, 4, 5, 6, 7, 8], dtype=int)
)
```

In [11]:

```
# -------------------------
# Assemble Global Stiffness System
# -------------------------
# This step assembles the global stiffness matrix and the global
# fixed-end force vector from all elements.

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )
```

In [12]:

```
# -------------------------
# Display Global Stiffness Matrix
# -------------------------
# Print the assembled global stiffness matrix in a readable format.
# The matrix is scaled for clearer presentation in the slides.

print_matrix_scaled(K_global, scale=1000, decimals=2, col_width=6)
```

```
K = 1e+03 ×
01 |  61.88 -17.12 -28.80 -38.40  -4.28  17.12 -28.80  38.40
02 | -17.12 170.88 -38.40 -51.20  17.12 -68.48  38.40 -51.20
03 | -28.80 -38.40  28.80  38.40   0.00   0.00   0.00   0.00
04 | -38.40 -51.20  38.40  51.20   0.00   0.00   0.00   0.00
05 |  -4.28  17.12   0.00   0.00   4.28 -17.12   0.00   0.00
06 |  17.12 -68.48   0.00   0.00 -17.12  68.48   0.00   0.00
07 | -28.80  38.40   0.00   0.00   0.00   0.00  28.80 -38.40
08 |  38.40 -51.20   0.00   0.00   0.00   0.00 -38.40  51.20
```

In [13]:

```
# -------------------------
# Partition Global System
# -------------------------
# The global stiffness equation is partitioned into free and restrained
# degrees of freedom based on the support conditions.

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [14]:

```
# -------------------------
# Solve Partitioned System
# -------------------------
# Solve for the unknown displacements at the free DOFs,
# compute the reactions at restrained DOFs, and then
# reconstruct the complete global displacement and force vectors.

rhs = f_f - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

F_r = K_rf @ u_f + K_rr @ u_r + f_fef_r

u_global = assemble_global_displacements(
    u_f,
    u_r,
    free_dofs,
    restrained_dofs
    )

f_global_complete = assemble_global_forces(
    f_f,
    F_r,
    free_dofs,
    restrained_dofs
    )
```

In [15]:
```
# -------------------------
# Display DSM Results
# -------------------------
# Print the final global displacement vector and force vector.

print_dsm_results(
    u_global,
    f_global_complete,
    dof_restrained_1based,
    disp_in_mm=True,
    member_type="truss"
)
```

```
 DOF Type Status  Disp (mm / rad)  Load (kN / kN·m)
   1  u_x   Free           8.920          600.000
   2  u_y   Free          -6.171         -900.000
   3  u_x  Fixed           0.000         -106.324
   4  u_y  Fixed           0.000         -141.766
   5  u_x  Fixed           0.000         -143.824
   6  u_y  Fixed           0.000          575.297
   7  u_x  Fixed           0.000         -349.852
   8  u_y  Fixed           0.000          466.469
```

Member-Level Solution (Local)

Then the backward pass goes extract u_global for the element

$$\mathbf{u}_i \leftarrow \mathbf{u}_{\text{global}}$$

Convert global DOFs to the element's local coordinate system:

$$\mathbf{v}_i = \mathbf{T}_i \, \mathbf{u}_i$$

Local element forces are the sum of displacement-related forces and FEFs:

$$\mathbf{q}_i = \mathbf{k}_{\ell,i} \, \mathbf{v}_i + (\mathbf{q}_i^F + \mathbf{q}_i^H + \mathbf{q}_i^E)$$

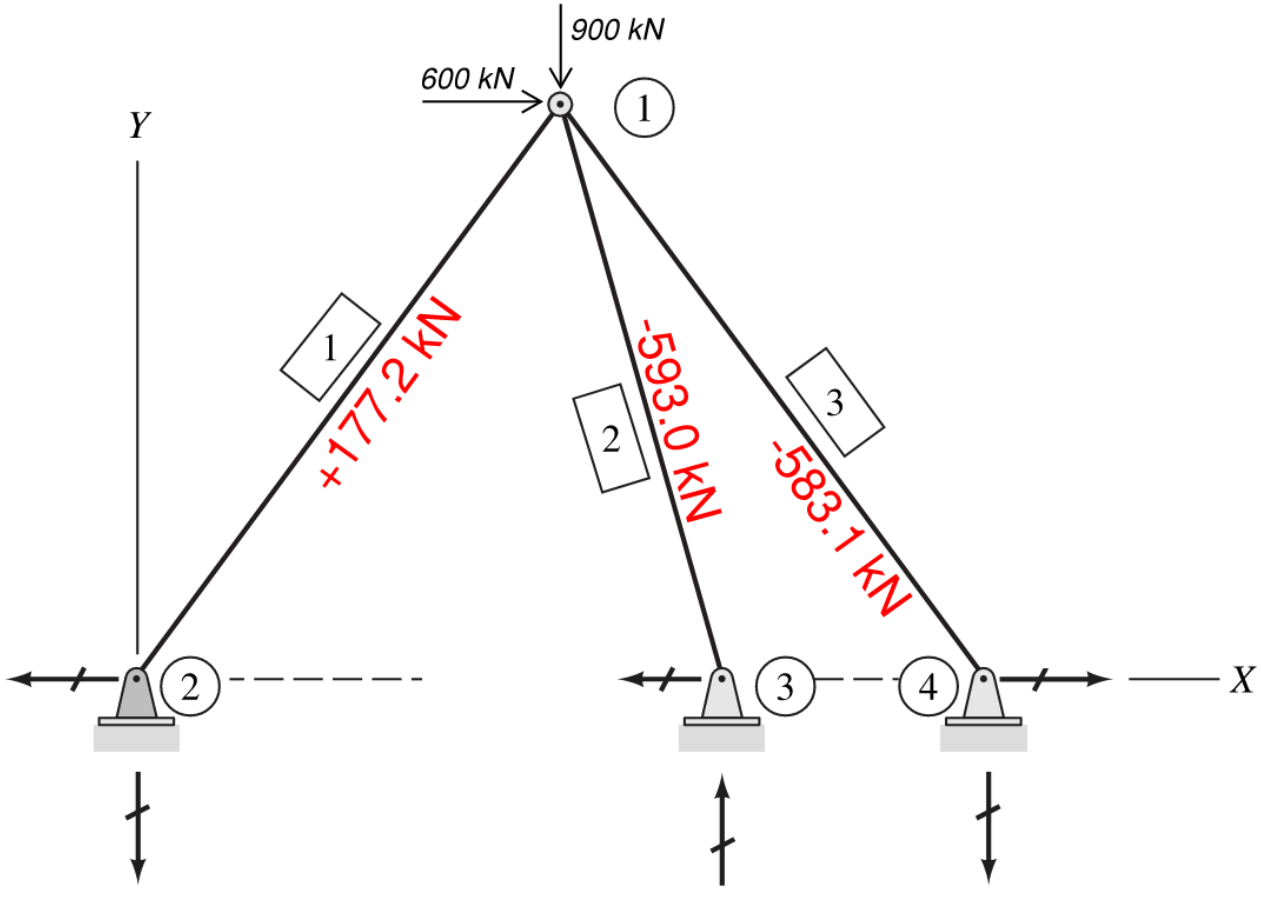
In [16]:
```
# -------------------------
# Display Element Results
# -------------------------
# Compute and print the element-level results for each member.

print_element_truss(1, u_global, m1, T1, k1, Qf1, disp_in_mm=True, dec=2)
print_element_truss(2, u_global, m2, T2, k2, Qf2, disp_in_mm=True, dec=2)
print_element_truss(3, u_global, m3, T3, k3, Qf3, disp_in_mm=True, dec=2)
```

```
E1 (Truss)
u_global [mm]: [0.00, 0.00, 8.92, -6.17]
u_local  [mm]: [0.00, 0.00, 0.42, -10.84]
q_local  [kN]: [-177.21, 0.00, 177.21, 0.00]
N (tension +) = 177.21 kN


E2 (Truss)
u_global [mm]: [0.00, 0.00, 8.92, -6.17]
u_local  [mm]: [0.00, 0.00, -8.15, -7.16]
q_local  [kN]: [593.00, 0.00, -593.00, 0.00]
N (tension +) = -593.00 kN


E3 (Truss)
u_global [mm]: [0.00, 0.00, 8.92, -6.17]
u_local  [mm]: [0.00, 0.00, -10.29, -3.43]
q_local  [kN]: [583.09, 0.00, -583.09, 0.00]
N (tension +) = -583.09 kN
```


900 kN
600 kN
1
Y
1
+177.2 kN
2
-593.0 kN
3
-583.1 kN
2
3
4
X

Member-Level Solution (Global)

Then transform **member end forces in local coordinates** to the **global coordinate system** using $f = T^T q$

The resulting global force components provide the **reactions at the supports** due to the prescribed support displacement.

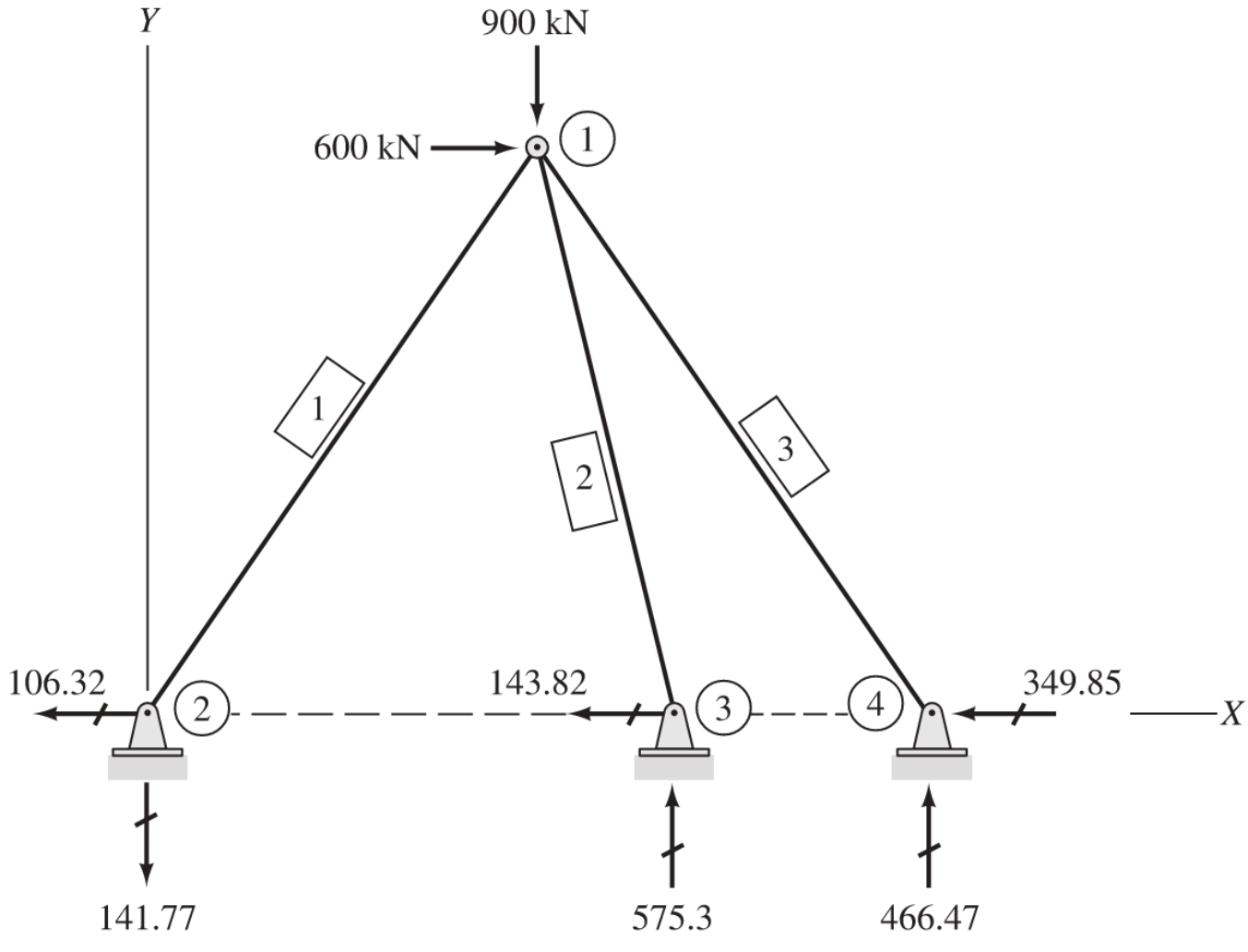
In [17]:
```
idx = m1 - 1
u = u_global[idx]
v = T1 @ u
q = k1 @ v + Qf1
f = T1.T @ q

print(np.round(f, 2))
```

```
[-106.32 -141.77  106.32  141.77]
```

All elements start at supports and end at top node.

First two terms in each element's f-vector give support forces:


Y
900 kN
600 kN
1
1
2
3
106.32
2
143.82
3
4
349.85
X
141.77
575.3
466.47

Wrap-Up

Temperature changes and fabrication errors are handled in DSM as **equivalent nodal force effects**.

For temperature changes:

- average temperature produces axial restraint force
- temperature gradient produces end moments

For fabrication errors:

- length error produces axial restraint force
- straightness error produces shear/moment restraint effects

At the element level:

$$\mathbf{q}_i = \mathbf{k}_{l,i}\mathbf{v}_i + (\mathbf{q}_i^F + \mathbf{q}_i^H + \mathbf{q}_i^E)$$

At the global level:

$$\mathbf{K}\mathbf{u} = \mathbf{f} - (\mathbf{f}^F + \mathbf{f}^H + \mathbf{f}^E)$$